

Computational Linguistics Project Report:

(<https://colab.research.google.com/drive/11Vmc-3eHRS3fwue0pnmqSFkCue3GxmfG#scrollTo=BNDUu2TtV4sP>)

The aim of this project was to perform sentiment analysis on a dataset of Youtube comments. The dataset contains 1500 rows, as well as two columns called “sentiment” and “comments”. It is a binary classification task, since there are two labels present in the dataset, namely “Nostalgia” and “Not Nostalgia”.

The models I used were google-bert/bert-base-cased as well as FacebookAI/xlm-roberta-base. With GoogleBert, I went for the case version so as to include things like all capitalizations, as I initially thought that could be useful for sentiment analysis in the social media sector. For me personally, it was very interesting to see how well these models would perform with such an emotion as “nostalgia” as I myself would probably find this classification a bit tricky.

Since this is a supervised learning task and the data has subsequently been labeled, I was curious as to how the dataset looked like and what was labeled as (not) nostalgic. I just find nostalgia a very difficult emotion to classify in general. For example, the following comment: “Absolutely Brilliant, thanks for posting this video for people who appreciate good music, yes I’m older now, but at least I can boast that I enjoyed the days when Jim was in his element. “, was classified as not nostalgic, which I would disagree with. This goes to show how subjective labelling is/can be.

As for other features of the dataset: It isn’t split into training and test set, but “merely” into training set, so I had to split it into training, test and validation set myself. I started with a very small training/validation/test set split (128/16/16) but eventually tried to incorporate the almost the entire dataset (1024, 128,128). However, while the training worked fine, I did run into some trouble while trying to load this model for visualization (as my runtime kept getting cut off) which is why I ended up downsizing again and my final dataset split contained 512 for training, 64 rows for validation as well as 64 rows for testing. The labels are distributed very evenly (50/50), as can be seen in HuggingFace.

I trained with GPU on Google Colab, which was one of the most nerve-racking parts, since at multiple points, I, again, kept encountering runtime errors. As for hyperparameters, I

did try to optimize my hyperparameter search with Optuna, however, it always led to my model not learning anything somehow (accuracy stayed the same continuously), which is why I stuck with simply experimenting with the hyperparameters. For the FacebookAI model, my initial hyperparameters ended up being the ones that performed the best for the validation set (up to 98% accuracy).

```
arguments = TrainingArguments(  
    output_dir="/content/drive/MyDrive/Colab Notebooks/cl_facebookai",  
    per_device_train_batch_size=16,  
    per_device_eval_batch_size=16,  
    logging_steps=8,  
    num_train_epochs=5,  
    eval_strategy="epoch",  
    save_strategy="epoch",  
    learning_rate=2e-5,  
    weight_decay=0.01,  
    load_best_model_at_end=True,  
    report_to='none',  
    seed=224  
)
```

Figure 1: Hyperparameters FacebookAI

Concerning GoogleBert, I tried the same parameters as FacebookAI's XLM-RoBERTa, but I ended up with a bit of a rollercoaster regarding the accuracy results during training, which is why I decided to lower the learning rate to 1e-5, which is when the accuracy stayed more consistent during the training. I then increased the number of training epochs to 6, to "make up" for the lower learning rate and ended up with accuracy results of 96.875% on the validation set.

```
arguments = TrainingArguments(  
    output_dir="/content/drive/MyDrive/Colab Notebooks/cl_google",  
    per_device_train_batch_size=16,  
    per_device_eval_batch_size=16,  
    logging_steps=8,  
    num_train_epochs=6,  
    eval_strategy="epoch",  
    save_strategy="epoch",  
    learning_rate=1e-5,  
    weight_decay=0.01,  
    load_best_model_at_end=True,  
    report_to='none',  
    seed=224  
)
```

Figure 2: Hyperparameters GoogleBert

Regarding the evaluation on the test set, the results were also a bit nerve-wracking at first glance, since I originally ended up with an accuracy score of 1 for both models, which was

highly suspicious. After ensuring that there was no overlap in between the training and test set, I then removed the truncation limit to let the models see all of the comment strings (which however had no impact on the evaluation results) in an effort to rule out all possibilities that could lead to such high accuracy. I also tried to see if there were any duplicates that might inflate the accuracy score, however I wasn't able to detect any. This 100% accuracy score is also why I opted for using every possible other evaluation score that I have learnt so far. I was curious as to if and how they differed from the accuracy score. However, after having imported the necessary metrics from scikit learn (precision, recall, F1-Score, MCC), the results stayed the same. After further feedback which, I eventually came to the conclusion that the dataset was just that simple (or rather, very clearly labelled, some of which I subjectively disagree with as I stated at the beginning of this report as well).

The very good evaluation across all evaluation metrics also made subsequent qualitative analysis extremely difficult, since the models apparently didn't have any problems identifying/generalizing the nostalgia in the dataset, which is why I decided to analyze said dataset further, to see what the words were most associated with nostalgia. The top relevant words I could discern for nostalgia were "memories", "reminds" and "remember". Only very few comments with these words in them were labelled as not nostalgic.

By chance, I found an example where the XLM-RoBERTa was "wrong". The sentence: "I remember my father used to listen to Jim Reeves when I was young. I am now 64." from the dataset was labelled "not nostalgic", however the model classified it as nostalgic, which is unsurprising, given that "remember" is one probably of the strongest indicators for the model that a comment is nostalgic. Similarly, the sentence "I was in 2nd grade when this movie came out. When you age, your memory is your only treasure. Build a treasure for your youth. ", was also classified as nostalgic (keyword being memory here) by the model, but originally labelled as not nostalgic. This makes me wonder if the labelers had a very specific definition of nostalgia in mind that I'm not quite grasping. However, in Hugging Face there was no documentation by the authors of how exactly the labelling was done/what definition was used in the dataset.

Since I could manually find a few instances where at least one of the models was wrong, I consequently tried increasing the test set (from 64 to 512 examples), which is when all the scores eventually went down a bit, but stayed at 98-99% accuracy, which indicates that the models are still performing extremely well, just with a few more edge cases/labeling inconsistencies showing up with the larger test size. This slight drop actually made the initial results feel more credible to me, since 100% accuracy on a larger test set would have been even more unlikely. The fact that both models are still hovering around 98-99% suggests that they've learned the general pattern really well, with the small errors probably coming from the cases where the labels themselves are questionable (like the examples I found with "remember" and "memory" that were labeled as not nostalgic). Therefore, I find it hard to say whether these are genuine model failures or just instances where the model is actually right and the original labeling was inconsistent. Again, this comes down to the subjectivity when labelling for sentiment analysis. Either way, the near-perfect performance confirms that this dataset is relatively easy for these models to handle, probably because the linguistic markers for nostalgia (like memory-related vocabulary or past tense descriptions) are for the most part pretty distinct and consistent throughout the dataset.

As for the visualization of the training process (using the test size of 64), the struggle lay in getting the colors correctly in `projector.tensorflow`, which meant adjusting the column names (since "label" is called "sentiment" in my dataset). This was a relatively quick adjustment to make. The actual visualizations are depicted down below. Figure 3 depicts the first layer of the training of FacebookAI's XLM-RoBERTa (but not layer 0, since everything is in one place at that point). At this stage, everything is still quite far apart and mixed together randomly. Figure 4 depicts the 12th layer, which is where the examples have already been split and are classified correctly. Here, it's also visualized how perfect the models classified the test dataset by the 12th layer, with no visible outliers.

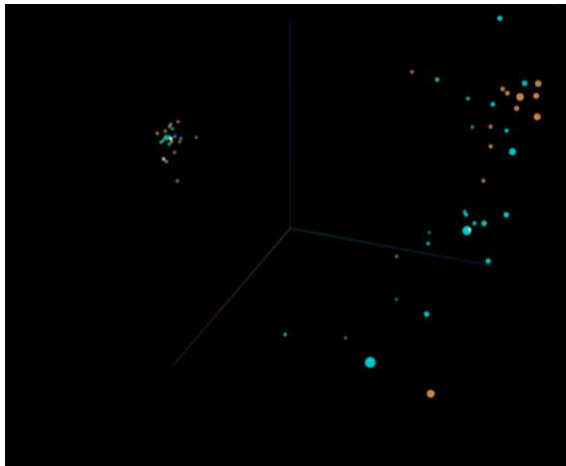


Figure 3: Layer 1 FacebookAI

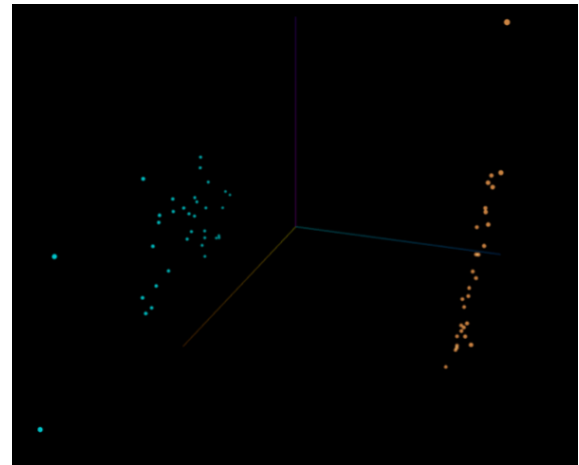


Figure 4: Layer 12 FacebookAI

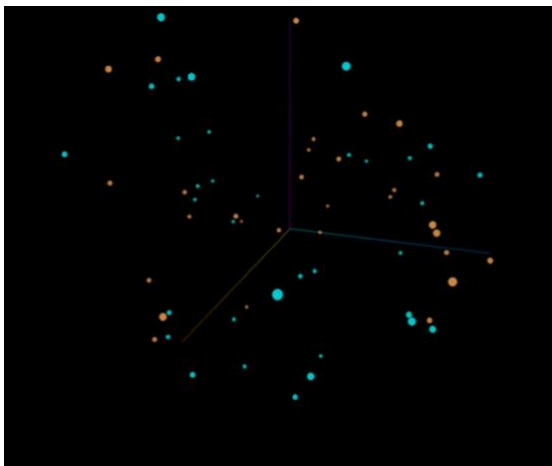


Figure 5: Layer 1 GoogleBert

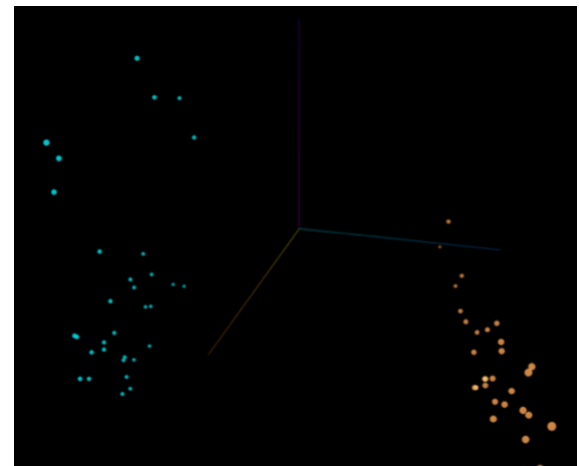


Figure 6: Layer 12 GoogleBert

The same can be observed in GoogleBert's case, again with no visible outliers by the last layer. The first layer does differ slightly, though (Figure 3 and Figure 5) in that the initial representations of the data are different. GoogleBert's embeddings (Figure 5) are really spread out across the 3D space, with nostalgic and not nostalgic comments scattered all over the place without any clear structure. Consequently, the model initially learns very different representations for each comment, maybe even focusing on very general (?) features like capitalization (which would make sense in the case of the cased version here). On the other hand, FacebookAI's XLM-RoBERTa (Figure 3) shows some points clustered together in one group already (with the other group being more spread out still), even though they're still mixed with both colors. This indicates to me that the model already kind of recognizes that these comments are semantically similar to each other, even before it really starts

distinguishing between nostalgic and not nostalgic. I do have some inkling as to why that could be: It could have to do with how they're pretrained (XLM-RoBERTa's: multilingual/GoogleBert: monolingual), which might give XLM-RoBERTa better initial semantic groupings, compared to GoogleBert. As stated, despite these different starting points, both models end up achieving perfect separation by layer 12. However, this might explain, why fewer hyperparameter adjustments were needed during the training in XLM-RoBERTa's case.

In summary, both the GoogleBERT-based and FacebookAI XLM-RoBERTa models performed exceptionally well on the sentiment analysis task. From a quantitative perspective, they both achieved up to 100% accuracy on the initial test set of 64 examples, with similarly high scores across all other metrics including precision, recall, F1-score and MCC. When I increased the test set size to 512 examples, the accuracy (and other scores) dropped slightly to 98-99%, which is still extremely high and suggests the models generalize well even with more data. From a qualitative perspective, the few instances where the models were "wrong" seemed to be cases where the original labeling was inconsistent rather than model failures, e.g. comments with nostalgic markers like "remember" were labeled as not nostalgic in the dataset, even though the models classified them as nostalgic. Finally, from a visual analysis perspective, while both models ended up perfectly separating the two classes by the final layer, the more interesting part was actually how they started out. GoogleBERT began with highly separated embeddings across 3D space, while XLM-RoBERTa started with clustered representations, indicating that XLM-RoBERTa's pre-training might have given it a better initial semantic "understanding".

As for my personal experience, at the beginning of the semester I was worried that it would take me a long time to finish the project, however it ended up taking shorter than I had anticipated, which could also be attributed to the fact that I found it quite fun to play around with. The most "difficult" part was getting GoogleColab to run long enough without crashing, or generally the waiting portion of this project.

If I were to choose a task/dataset/model again, I do think I would choose a different task (or at least a different dataset that would have been more interesting to analyze). As for models, I went for the case-model of GoogleBert, since I thought that that would be beneficial

for social media comments (using capitalization to express excitement), however, in hindsight, that might not be the case for nostalgic content. So, I would maybe try with the uncased version to see how the visualization might differ in the first layer, too.

Sources:

Dataset:

Postalcioglu, S., & Aktas, S. (2020). Comparison of neural network models for nostalgic sentiment analysis of YouTube comments. Hittite Journal of Science and Engineering, 7(3), 215–221.

([https://huggingface.co/datasets/Senem/Nostalgic Sentiment Analysis of YouTube Comments Data](https://huggingface.co/datasets/Senem/Nostalgic_Sentiment_Analysis_of_Youtube_Comments_Data))

Models:

google-bert/bert-base-cased : <https://huggingface.co/google-bert/bert-base-cased>

FacebookAI/xlm-roberta-base: <https://huggingface.co/FacebookAI/xlm-roberta-base>